

Configurazione del Raspberry Pi per l'acquisizione di audio da microfono USB

Lucrezia Grassi

2 maggio 2026

1. Connessione alla rete e impostazione IP statico

Dopo l'accensione del Raspberry Pi, connettersi alla rete Wi-Fi locale alla quale sarà collegato anche il robot. Impostare un indirizzo IP statico attraverso il file di configurazione della rete oppure tramite l'interfaccia grafica, se disponibile.

È importante che il Raspberry Pi e il robot siano sulla stessa rete e che l'indirizzo IP del Raspberry Pi sia stabile, in modo che il robot possa connettersi sempre allo stesso indirizzo.

2. Scaricamento del codice

La guida assume che la repository venga salvata nella home dell'utente `pi`, quindi nel percorso:
`/home/pi/cair-audio-processing`

Per evitare errori nei passaggi successivi, si consiglia di clonare la repository direttamente nella cartella `/home/pi/`:

```
cd /home/pi
git clone https://github.com/lucregrassi/cair-audio-processing.git
```

In alternativa, è possibile scaricare il file ZIP della repository GitHub:

```
https://github.com/lucregrassi/cair-audio-processing
```

In questo caso, estrarre la cartella ZIP nella home dell'utente `pi` e verificare che il percorso finale sia:

```
/home/pi/cair-audio-processing
```

Se la repository viene salvata in un percorso diverso, tutti i comandi della guida che contengono:
`/home/pi/cair-audio-processing`

devono essere modificati di conseguenza.

3. Creazione del file `.env` con la chiave Azure

All'interno della cartella della repository, creare un file chiamato `.env`:

```
nano /home/pi/cair-audio-processing/.env
```

Inserire al suo interno la chiave Azure Speech nel seguente formato:

```
AZURE_SPEECH_KEY=la_tua_chiave
```

La chiave si ottiene creando un account su Microsoft Azure e seguendo la procedura per la creazione di una nuova risorsa *Speech Services*.

4. Disabilitare il risparmio energetico USB

Modificare il file di configurazione di avvio:

```
sudo nano /boot/firmware/cmdline.txt
```

Alla fine della riga, tutto su una riga sola e senza andare a capo, aggiungere:

```
usbcore.autosuspend=-1
```

Salvare il file. La modifica sarà applicata al riavvio successivo.

5. Aggiornare il sistema e installare le librerie audio

Eseguire l'aggiornamento dei pacchetti e installare o reinstallare ALSA:

```
sudo apt update
sudo apt install --reinstall alsa-utils libasound2 libasound2-dev
```

Installare inoltre le librerie di sistema necessarie per l'uso del microfono e per la compilazione di alcune dipendenze Python:

```
sudo apt install python3-venv python3-full python3-dev build-essential portaudio1
```

6. Configurare ALSA per il microfono USB

Verificare la posizione del microfono USB con:

```
arecord -l
```

Il comando mostrerà l'elenco dei dispositivi audio disponibili. Annotare il numero della scheda e del dispositivo associati al microfono USB.

Creare o modificare il file `.asoundrc` nella home directory:

```
nano /home/pi/.asoundrc
```

Inserire il seguente contenuto:

```
pcm.!default {
    type plug
    slave.pcm "hw:0,0"
}

ctl.!default {
    type hw
    card 0
}
```

Se il microfono non è sulla scheda 0, sostituire `hw:0,0` con il valore corretto, ad esempio:

```
hw:1,0
```

In tal caso, aggiornare anche il campo:

```
card 0
```

con il numero corretto della scheda.

7. Configurare l'ambiente Python

1. Posizionarsi nella cartella `cair-audio-processing` scaricata da GitHub:

```
cd /home/pi/cair-audio-processing
```

2. Creare un ambiente virtuale all'interno di questa cartella:

```
python3 -m venv venv
```

3. Attivare l'ambiente virtuale:

```
source venv/bin/activate
```

4. Aggiornare **pip**, **setuptools** e **wheel**:

```
python -m pip install --upgrade pip setuptools wheel
```

5. Installare le dipendenze del progetto:

```
python -m pip install -r requirements.txt
```

Se durante l'installazione di una libreria compare un errore relativo a un ambiente Python gestito esternamente, utilizzare sempre il Python dell'ambiente virtuale. Ad esempio:

```
/home/pi/cair-audio-processing/venv/bin/python -m pip install nome_pacchetto
```

8. Testare il funzionamento manualmente

Prima di configurare l'avvio automatico, è consigliabile testare manualmente il programma.

Eseguire:

```
cd /home/pi/cair-audio-processing  
/home/pi/cair-audio-processing/venv/bin/python audio_recorder.py
```

Se il programma parte correttamente, interromperlo con:

CTRL+C

Se compaiono errori relativi a pacchetti mancanti, installarli usando sempre il Python dell'ambiente virtuale:

```
/home/pi/cair-audio-processing/venv/bin/python -m pip install nome_pacchetto
```

9. Avvio automatico del microfono con systemd

Per avviare automaticamente il microfono all'accensione del Raspberry Pi, è consigliato utilizzare **systemd**.

La repository contiene già il file **start_microphone.sh**. Non è quindi necessario crearlo da zero. È però importante verificare che il file si trovi nel percorso previsto dalla guida e che, al suo interno, i percorsi siano corretti (la prima volta sarà necessario sostituire **rice** con **pi** nei percorsi).

La guida assume che la repository si trovi in:

```
/home/pi/cair-audio-processing
```

Prima di configurare il servizio, verificare che lo script esista nel percorso previsto:

```
ls /home/pi/cair-audio-processing/start_microphone.sh
```

Se il comando restituisce un errore, significa che la repository si trova in un percorso diverso. In questo caso, aggiornare tutti i percorsi presenti nello script **start_microphone.sh** e nel file **start-microphone.service**.

1. Aprire lo script **start_microphone.sh**:

```
nano /home/pi/cair-audio-processing/start_microphone.sh
```

2. Verificare che i percorsi presenti nello script coincidano con il percorso effettivo della repository.

Se la repository si trova in:

```
/home/pi/cair-audio-processing
```

lo script dovrebbe usare percorsi coerenti con questo, ad esempio:

```
cd /home/pi/cair-audio-processing
exec /home/pi/cair-audio-processing/venv/bin/python /home/pi/cair-audio-proc
```

Se la repository si trova in un percorso diverso, modificare tutte le occorrenze di `/home/pi/cair-audio-pro` nello script.

Verificare inoltre che il comando Python non termini con il simbolo `&`. Il processo non deve essere mandato in background, perché deve essere monitorato direttamente da `systemd`.

3. Rendere lo script eseguibile:

```
chmod +x /home/pi/cair-audio-processing/start_microphone.sh
```

4. Creare un servizio `systemd`:

```
sudo nano /etc/systemd/system/start-microphone.service
```

5. Inserire il seguente contenuto:

```
[Unit]
Description=Start microphone script
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/cair-audio-processing
ExecStart=/home/pi/cair-audio-processing/start_microphone.sh
Restart=on-failure
RestartSec=5
StandardOutput=append:/home/pi/audio_recorder_log.txt
StandardError=append:/home/pi/audio_recorder_log.txt

[Install]
WantedBy=multi-user.target
```

Anche il file di servizio `systemd` deve contenere gli stessi percorsi usati nello script. Se la repository non si trova in `/home/pi/cair-audio-processing`, modificare i campi `WorkingDirectory`, `ExecStart`, `StandardOutput` e `StandardError`.

6. Ricaricare `systemd`:

```
sudo systemctl daemon-reload
```

7. Abilitare il servizio all'avvio:

```
sudo systemctl enable start-microphone.service
```

8. Avviare subito il servizio:

```
sudo systemctl start start-microphone.service
```

9. Controllare lo stato del servizio:

```
systemctl status start-microphone.service
```

Se il servizio è avviato correttamente, dovrebbe comparire lo stato:

```
active (running)
```

Se invece compare:

```
inactive (dead)
```

significa che lo script è terminato. In questo caso controllare che nel file `start_microphone.sh` il comando Python non termini con `&` e che i percorsi siano corretti.

10. Controllare i log del servizio

Per visualizzare i log del servizio in tempo reale:

```
journalctl -u start-microphone.service -f
```

Se è stato configurato il file di log indicato nella sezione precedente, è possibile controllarlo con:

```
tail -f /home/pi/audio_recorder_log.txt
```

Per visualizzare gli ultimi messaggi senza seguire il log in tempo reale:

```
journalctl -u start-microphone.service -n 100 --no-pager
```

11. Riavvio del Raspberry Pi

Dopo aver configurato il servizio, riavviare il Raspberry Pi:

```
sudo reboot
```

12. Controllo finale dopo il riavvio

Dopo il riavvio, verificare che il servizio sia attivo con:

```
systemctl status start-microphone.service
```

Per controllare che il processo Python sia attivo, è possibile usare:

```
htop
```

oppure:

```
ps aux | grep audio_recorder.py
```

13. Comandi utili per gestire il servizio

Per fermare il servizio:

```
sudo systemctl stop start-microphone.service
```

Per riavviare il servizio:

```
sudo systemctl restart start-microphone.service
```

Per disabilitare l'avvio automatico:

```
sudo systemctl disable start-microphone.service
```

Per riabilitare l'avvio automatico:

```
sudo systemctl enable start-microphone.service
```

14. (Opzionale) Connessione a una rete Wi-Fi da shell

Per connettere il Raspberry Pi a una nuova rete Wi-Fi tramite shell, è possibile utilizzare i seguenti comandi.

Per visualizzare tutte le reti Wi-Fi disponibili:

```
nmcli device wifi list
```

Per connettersi a una rete:

```
sudo nmcli device wifi connect "NOME_SSID" password "PASSWORD_WIFI"
```

Sostituire NOME_SSID e PASSWORD_WIFI con il nome e la password della nuova rete.

15. (Opzionale) Configurazione dei DNS

Se il Raspberry Pi ha problemi di risoluzione dei nomi di dominio, è possibile impostare DNS pubblici.

Per esempio, con `nmcli`, verificare il nome della connessione:

```
nmcli con show
```

Poi impostare DNS Cloudflare:

```
sudo nmcli con mod "NOME_CONNESSIONE" ipv4.dns "1.1.1.1"  
sudo nmcli con mod "NOME_CONNESSIONE" ipv4.ignore-auto-dns yes  
sudo nmcli con up "NOME_CONNESSIONE"
```

Sostituire NOME_CONNESSIONE con il nome effettivo della connessione.

Per verificare:

```
resolvectl status
```

Conclusione

A questo punto, il Raspberry Pi è configurato per mettersi in attesa della connessione da parte del robot e iniziare ad acquisire audio appena sul robot viene avviato il client CAIR e viene stabilita la connessione socket tra i due dispositivi.

La guida assume che la repository si trovi nel percorso:

```
/home/pi/cair-audio-processing
```

Se si usa un percorso diverso, è necessario aggiornare coerentemente tutti i percorsi nello script `start_microphone.sh`, nel servizio `start-microphone.service` e nei comandi di test.

Se il programma termina con errore, il servizio `systemd` proverà a riavviarlo automaticamente. Se invece la connessione con il client viene persa ma il programma continua a funzionare, il software si rimetterà in attesa di connessione con il robot, senza bisogno di riavviare il Raspberry Pi.